SOFTWARE DEV

MERN STACK DEVELOPMENT REPORT

Week 4 : Asynchronous JavaScript, APIs, and Modular Architecture

Muneeb Ur Rehman

MERN STACK INTERN

https://github.com/955muneeb/Glaxit-internship/tree/main/week-4

Laboratory Report – Week 5, Day 3

The React Lifecycle & useEffect Hook

Abstract

This laboratory exercise focused on understanding the React component lifecycle and the **useEffect** hook. The main goal was to learn how React components perform tasks automatically after they are displayed on the screen. We created a component that fetches mock post data from an online API as soon as it mounts. During the process, a loading spinner was displayed while waiting for the data, and any errors were handled by showing a friendly message to the user. This lab demonstrated how React applications communicate with external APIs while providing a smooth user experience.

Objectives

The objectives of this lab were:

- To understand the React component lifecycle.
- To learn how the **useEffect** hook works.
- To fetch data automatically when a component mounts.
- To manage loading and error states using **useState**.
- To display fetched data in the user interface.
- To understand the importance of cleanup functions in React.

React Component Lifecycle

Every React component goes through three stages known as the **component lifecycle**.

Mounting

Mounting is the first stage when a component is created and displayed on the screen for the first time. This is the best time to fetch data from an API because the component is ready to display the information once it is received.

Updating

Updating happens whenever the component's state or props change. React automatically re-renders the component so that the latest information is shown on the screen.

Unmounting

Unmounting is the final stage when the component is removed from the screen. Before removing it, React allows us to clean up resources such as timers or unfinished API requests to avoid unnecessary warnings or memory leaks.

Understanding useEffect

The **useEffect** hook is used to perform side effects after a component has been rendered. Side effects include tasks such as fetching data, making API calls, setting timers, or listening to browser events.

The basic syntax is:

```
useEffect(() => {
   // Side effect
}, []);
```

The empty dependency array `[]` tells React to run the effect only once after the component is mounted. This is why it is commonly used for fetching data from an API.

Task Performed

The task was to create a **PostsList** component that automatically loads placeholder posts from the **JSONPlaceholder API** when the component appears on the screen.

Three state variables were created:

- **posts** – stores the data received from the API.
- **loading** – keeps track of whether the data is still loading.
- **error** – stores an error message if something goes wrong.

The API request was written inside the **useEffect** hook using an asynchronous function.

The request was wrapped inside a **try-catch-finally** block:

- The **try** block fetched the data and converted it into JSON format.
- The **catch** block handled network or server errors and stored an error message.
- The **finally** block changed the loading state to **false**, ensuring that the spinner disappeared whether the request succeeded or failed.

Displaying the User Interface

The component checked its state before deciding what to display.

- If **loading** was `true`, the **Spinner** component was displayed.
- If an error occurred, a friendly error message was shown.
- If the data was successfully fetched, the posts were displayed in a grid layout.

This approach made the application more user-friendly because users always knew what was happening instead of seeing a blank screen.

Cleanup Function

A cleanup function was also added inside **useEffect** using an **isCancelled** flag.

If the user left the page before the API request finished, React would normally try to update the component after it had already been removed. This could produce warnings in the browser console.

The cleanup function prevents these unnecessary state updates, making the application safer and more reliable.

Components Created

The following components were created during this exercise:

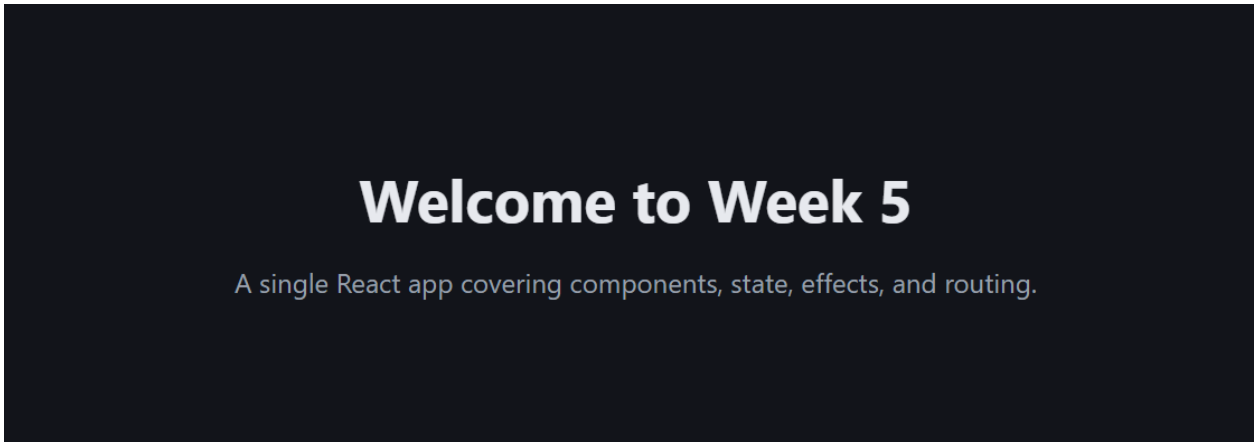
- **PostsList.jsx** – Fetches data from the JSONPlaceholder API, manages loading and error states, and displays the posts.
- **Spinner.jsx** – Displays a loading spinner while waiting for the API response.

What We Learned

This lab helped us understand how the React lifecycle works and why the **useEffect** hook is important. We learned that the dependency array controls when an effect runs. An empty dependency array `[]` runs the effect only once after mounting, while omitting it causes the effect to run after every render.

We also learned that handling **loading** and **error** states separately makes applications more reliable and provides a better user experience. Finally, we understood the purpose of cleanup functions and how they prevent unwanted state updates when a component is unmounted.

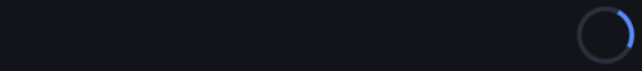
Screenshot



Welcome to Week 5

A single React app covering components, state, effects, and routing.

Latest Posts



Latest Posts

Sunt Aut Facere Repellat Provident Occaecati Excepturi Optio Reprehenderit

quia et suscipit suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet architecto

Qui Est Esse

est rerum tempore vitae sequi sint nihil reprehenderit dolor beatae ea dolores neque fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis qui aperiam non debitis possimus qui neque nisi nulla

Ea Molestias Quasi Exercitationem Repellat Qui Ipsa Sit Aut

et iusto sed quo iure voluptatem occaecati omnis eligendi aut ad voluptatem doloribus vel accusantium quis pariatur molestiae porro eius odio et labore et velit aut

Eum Et Est Occaecati

ullam et saepe reiciendis voluptatem adipisci sit amet autem assumenda provident rerum culpa quis hic commodi nesciunt rem tenetur doloremque ipsam iure quis sunt voluptatem rerum illo velit

Nesciunt Quas Odio

repudiandae veniam quaerat sunt sed alias aut fugiat sit autem sed est voluptatem omnis possimus esse voluptatibus quis est aut tenetur dolor neque

Dolorem Eum Magni Eos Aperiam Quia

ut aspernatur corporis harum nihil quis provident sequi mollitia nobis aliquid molestiae perspiciatis et ea nemo ab reprehenderit accusantium quas voluptate dolores velit et doloremque molestiae

```
try {
  setLoading(true)
  setError(null)
  const response = await fetch('https://jsonplaceholder.typicode.com/

  if (!response.ok) {
    throw new Error(`Request failed with status ${response.status}`)
  }

  const data = await response.json()
  if (!isCancelled) {
    setPosts(data)
  }
} catch (err) {
  if (!isCancelled) {
    setError(err.message || 'Something went wrong while fetching post
```